Intoxication Detection

Ekaterina Kolos & Laura Zeidler

{st186032|st186012}@stud.uni-stuttgart.de

Abstract

In this paper we describe our project for the class "Computational Linguistics Team Laboratory: Phonetics". We implemented two types of neural models, namely a simple neural network architecture and an LSTM architecture, to identify intoxicated speech from the ALC dataset. In order to improve the performance of these models , we adopted different strategies while training e.g. balancing the data and global speaker normalisation. Our final results suggest that normalising the speech data can significantly improve performance, while data balancing is only helpful to a certain extend.

1. Introduction

Intoxication detection, the task of determining whether a person is intoxicated solely by means of their speech, has emerged as a challenging problem with significant implications for public safety. Being able to tell intoxicated and sober speech apart automatically could for instance help with reducing accidents caused by drunk driving [1].

Intoxicated speech, as compared to sober speech, is characterised by "disfluencies such as repetitions, false starts, word and sentence breaks, changes in speaking rate, increased rate of unfilled and filled pauses, unusual prolongations of phones, and filler sounds" [2]. However, the grade of individual variation in how intoxicated speech is different from sober is high, and different speech act types (e.g. long monologues as opposed to short commands) are disrupted to different extent [2], thus making the task of intoxicated speech detection non-trivial even for humans: for example, a study of 79 individuals trying to identify intoxication on a balanced intoxicated/non-intoxicated sample set are reported to have achieved an average performance of only 55.8% unweighted accuracy, with the best individual achieving UA of 70% [3].

Furthermore, there is no consistent evidence of whether intoxication is manifested in a certain way within particular **groups of speakers**. On the contrary, Hollien et al. (2001) come to the conclusion that neither the gender nor the drinking habit profile play a significant role in perceived intoxication [4] .

This falls in line with the empirical study by Schiel at al. (2015): "The gender factor was never found to interact with intoxication" [2]. At the same time, another extensive study of the effects of gender, topic, age etc on the disfluency in intoxicated speech finds out that "older speakers produced only slightly higher disfluency rates than young and middle-aged speakers" [5]. Taking all this into account we decide not to explore metadata related differences and focus primarily on the acoustic input in our work.

There have been a few **previous studies** on automating intoxicated speech detection. Aishwarya et al. (2023) construct their own dataset based on TV extracts and compare MFCC-based classifiers and CNNs for intoxicated speech detection, achieving good accuracy in their setting [6]. A larger thread of work has been done on the corpora introduced in the Interspeech Speaker State challenge [7], with the baseline for intoxication detection set by a linear SVM model on data balanced using WEKA achieving an unweighted accuracy slightly above 65%. Schiel et al. (2010) explore the statistical importance of rhythm and formant features in predicting intoxication in speech, highlighting the impact of the former [8]. Hönig et al. (2011) show that prosodic features actually contribute additional information to SOTA features. They train an SVM model on the enriched representation with an improvement of around 2% over the baseline, achieving an unweighted accuracy of 67.6% in their best setting [9]. Other modifications of the original approach have shown small improvements, such as lingustic features and feature selection getting to 66.6% UA [3], doing ensemle learning including phoneme-based classifiers, with the best average UA of 71.4% [10], while boosting the performance with spectral-based systems using MFCCs, PLPs, TRAPS achieves up to 73.6 % UA [11]. [12] achieve an unweighted accuracy of 66.2%, experimenting with SVM, SMO and noise cancelling on OpenSMILE features.

In our work, we tackled this problem by investigating multiple approaches to improve neural network performance on the task of intoxication detection in speech, such as normalising the data or balancing the data at hand.

We formalize the task as *binary classification* of whether the speaker is intoxicated or not, consistent with the labels of the annotated corpus we use (i.e. 'intoxicated' if the blood alcohol level is between 0.5 ‰ and 2.5 ‰). Alternatively, the regression task of predicting the level of blood/breath alcohol concentration could be solved [1].

2. Data

The data that was used in this work is the Alcohol Language Corpus by [13]. This corpus includes well documented intoxicated as well as sober speech. It contains in total 14,580 recordings from 162 speakers. Participants were recorded in a car where they were prompted with different instructions, like reading a tongue twister, giving the car commands to get to a certain address or just a prompt for a mono- or dialogue. Each participant was recorded in both sober and intoxicated condition, the degree of intoxication was controlled by measuring the

[1] We also attempted a regression analysis, that showed weak correlation of individual features with the label, with the most correlated feature being `38: logRelF0-H1-A3_sma3nz_amean`

blood alcohol concentration and the breath alcohol concentration. The ALC corpus includes the speech recordings as well as the corresponding orthographic transcripts and phonemic segmentation, Praat annotations and meta annotations concerning the speaker or some external factors of the recording (e.g. the weather condition during the recording).

3. Preprocessing

3.1. Data Balancing

The dataset is constructed in such a way that for every speaker there should be twice as many sober recordings as intoxicated ones, except for 7 speakers who were recorded only in the intoxicated state. Furthermore, there's a different number of recordings for each `content` (A/P/Q/N/R/C/S/T = Address / Picture / Question / Number / Read Command / Spont. Command / Spelling / Tongue twister) and `type` (R/E/M/D/L = Read / Elicited / Monologue / Dialogue / List), and these do not form a hierarchy. We created two kinds of balanced datasets: first, 50-50 balanced dataset, that aims to just have the same amount of intoxicated recordings as sober recordings; second, two per-speaker balanced datasets that aim to have a small but equal amount of recordings for every `content` or `type` subgroup.

3.2. Global Speaker Normalisation

In [14] the authors find that speaker normalisation increases performance for the task of intoxication detection. They use multiple types of normalisation, but their results suggest that global speaker normalisation, which is computed by taking into account the values for each feature across all utterances from a speaker, indifferent to the class-label distribution, works best. [14] use Z-normalisation, where the difference between the original feature value f_{orig} and the mean μ of that feature is divided by the feature's standard deviation σ:

$$f_{new} = (f_{orig} - \mu)/\sigma$$

This normalisation step ensures that the mean of all of the values of a specific feature is 0 and the standard deviation is 1.

4. Methods

4.1. Experimental Setup

The original dataset was split into train-validation-test partitions (80-10-10). We trained our models on the train partition, as well as on each of the modified datasets where balancing and/or normalization were applied. To understand what approach works better and be able to compare the results adequately, we evaluated all of our different models on the validation partition of the 50/50 data split. The reason for choosing the 50/50 split over the original data split for this was that the unbalanced class distribution in the original dataset makes it harder to see whether a model actually learned or improved because the accuracy can be very high from just predicting the most common class anyway.

Normalized models were only tested on normalized data, hence the normalised 50/50 validation split. We further made sure that no overlap would occur between the different training data splits and the 50/50 validation and test splits. Only the best model from each training condition, i.e. training dataset, was tested on the actual test split.

4.2. Baselines

For each of the experimental setup settings, we ran the following baselines: a majority baseline, which simply assigns the most frequently occurring class to a new test sample; several supervised ML models (`sklearn` implementation [15]), based on *Functional* features. Two types of models were learnt: based on normalized and not normalized values. Moreover, a simple Transformer model was implemented in `PyTorch` based on *LLD* features, which only resulted in an accuracy of 48.868 (i.e. more or less random) on the datasets with most balancing (speaker-type and speaker-content balancing).

Table 1 summarizes the performance of the simple models in each experimental setting on Functional features.

Out of the classical supervised ML approaches for binary classification that we tried: Naive Bayes, kNN, SVM, Logistic Regression, Random Forest - it was the latter that consistently showed the best results, achieving, in the best case scenario (training on the largest balanced, i.e. '50-50' dataset and evaluated its validation partition) an accuracy of 0.698. The weighted accuracy, that this model achieves when evaluating on the validation partition of the original dataset, is 0.749. The second good performing model was SVM with a poly kernel, degree=3, followed by Logistic Regression. Basically all the baseline models surpassed the average performance of humans on this task. More ML baselines evaluations can be found in the appendix in table 11. [2]

Normalizing the features consistently helped improve or at least did not decrease the performance for all models except Random Forest, whose performance was sometimes lower with normalization. This holds for the comparable experiments on the 50-50 balanced subset, and is also observed in the experiments with different distributions of target classes in table 1. We speculate that Random Forest actually relies on one particular feature and bases its predictions on outlier values, which are lost at normalization; in the meantime, other baseline ML models are more likely to benefit from normalization because they take the whole vector representation of the signal into account. The models that could benefit the most from normalization were Logistic Regression, whose performance went up from 0.57 to 0.647 when train on the original dataset, and kNN, with up to 10% improvement (see speaker balanced settings).

4.3. Neural Models

The two types of features that we extracted from the data differ mostly in the fact that the functional features are single values while the LLD features are a sequence of values. Hence, these feature types cannot adequately be used by the same neural model. For this reason, we chose to build different architectures for the different feature types, namely a simple neural network (SNN) for the functional features and a Long Short-term Memory (LSTM) network for the LLD features.

Both networks were created using `PyTorch` [16]. They both use sigmoid as activation and apply a softmax in the output layer. The architectures are shown in Figure 3 and 4. Dotted lines represent steps within the architecture that are optional,

[2] these include evaluating the models trained on different subsets on the original validation partition ('full', 'speaker-content' and 'speaker-type' settings) or on the partition from the same distribution as the training data ('50-50', '50-50 norm.', 'speaker-content norm.', 'speaker-type norm.' settings, as described in table 10). The models were later run on the '50-50' validation partition to provide results consistent with the neural experiments.

approach	full		50-50 split		speaker-type		speaker-content	
	-	norm.	-	norm.	-	norm.	-	norm.
Majority Baseline	0.548	0.548	0.548	0.548	0.548	0.548	0.548	0.548
Naive Bayes	0.576	0.626	0.585	0.634	0.593	0.63	0.593	0.63
kNN	0.559	0.616	0.558	0.633	0.515	0.614	0.515	0.614
SVM (linear)	0.565	0.637	0.555	**0.664**	0.56	0.635	0.45	0.639
SVM (poly)	**0.606**	0.633	**0.616**	0.662	0.527	**0.657**	0.527	**0.657**
Logistic Regression	0.57	**0.647**	0.611	0.663	**0.6**	0.638	**0.6**	0.638
Random Forest	**0.655**	**0.652**	**0.698**	**0.669**	**0.655**	**0.652**	**0.643**	**0.653**

Table 1: *Baseline results*

the three dots imply n linear layers which are optional as well (i.e. only one linear layer is required).

4.4. Hyperparameter Search

In order to optimize our model performance, we decided to tune the models' hyperparameters. Table 5 and 6 show the hyperparameter combinations for the two model architectures. Since the LSTM takes much longer to train than the SNN and we wanted to keep time and computational resources within a reasonable frame, we only examined half as many hyperparameter combinations for the LSTM.

The strategy for choosing the hyperparameter combinations was simply to start with a random initial combination of hyperparameters and then changing one of the hyperparameters. If this change proved to be helpful[3], it would be adapted for the following combinations.

For the learning rate we always used the Toucan Warmup Scheduler by [17] in order to introduce a dynamic learning rate. Since changing the number of epochs, the learning rate or the optimiser did not improve performance for the SNN models, we did not experiment with these hyperparameters for the LSTM models. All LSTM models were therefore trained for 10 epochs with a learning rate of 0.0001 using Adam optimiser.

5. Results

The results for the experiments conducted with SNNs can be found in Table 2, 3 and 7, experiments with LSTMs in Table 4 and 8. The accuracies that were obtained on the test split of the data are reported in Table 9.

Param. combo	50/50 split	50/50 split (norm.)
combo 0	60.185	66.152 (+5.97)
combo 1	60.905	66.77 (+5.86)
combo 2	61.008	64.609 (+3.6)
combo 3	**62.654**	66.255 (+3.6)
combo 4	45.165	45.165 (+0.0)
combo 5	59.671	67.078 (+7.41)
combo 6	54.012	65.844 (**+11.83**)
combo 7	61.934	66.461 (+4.53)
combo 8	58.539	66.358 (+7.82)
combo 9	59.362	63.169 (+3.81)
combo 10	60.802	66.872 (+6.07)
combo 11	59.774	**68.107** (+8.33)

Table 2: *Accuracy of the SNN variations trained and tested on the 50/50 data split.*

[3]Based on the performance on the regular 50/50 split.

Combo	bal. for C	bal. for C (norm.)	bal. for T	bal. for T (norm.)
combo 0	57.407	63.477	54.835	60.391
combo 1	44.959	64.918	44.753	63.066
combo 2	45.165	62.346	49.486	61.008
combo 3	46.605	63.169	50.206	60.185
combo 4	45.165	54.835	54.835	53.909
combo 5	58.745	63.889	58.23	62.551
combo 6	54.733	62.551	51.029	61.42
combo 7	45.576	62.963	45.062	62.243
combo 8	57.51	62.243	53.086	62.346
combo 9	47.119	**65.021**	55.247	64.403
combo 10	**59.156**	64.095	**59.053**	**64.918**
combo 11	54.835	63.477	57.716	63.066

Table 3: *Accuracy of the SNN variations on the balanced 50/50 data splits.*

5.1. Analysis

First and foremost, it can be said that only one of our best models can exceed the accuracy obtained by Random Forest which outperforms the neural models for every training condition but the normalised regular 50/50 split. Also, most other baselines outperform even our best models for the other training conditions.

In Table 2, it can clearly be observed that the global speaker normalisation improves the performance of almost all the different models, the only exception being `combo 4`, which introduces a dropout of 0.5. All other hyperparameter combinations benefit significantly from the normalisation. This is also true for most of the models trained on the subsets balanced for content or type, with `combo 4` being the exception for type. Further, Figure 1 and 2 show that the normalisation helps the model predict intoxicated speech correctly, while only slightly affecting the prediction of sober speech. About the SNN models that were trained on the subsets, it can generally be said that they perform worse than the regular 50/50 split.

Generally, the LSTM models perform much worse than the SNNs, most of them even perform worse than the majority baseline. To what regards the global speaker normalisation, a similar trend can be observed for the LSTM models. Table 4 shows that normalisation is helpful for most models, however, there are some models whose performance actually suffers from applying normalisation (e.g. `combo 2` for the regular 50/50 split). The effect that balancing the data according to content or type has on the performance is not as clear for the LSTM models as for the SNN models. Some hyperparameter combinations actually profit from training on these smaller subsets, however, others do not.

The results from the experiments conducted on the unbalanced training set, documented in Table 7 and 8, show that there is very little variation in the performances of the different models for both model architectures. This is due to the fact that for most hyperparameter combinations, the model just chooses the

Combo	50/50 split	50/50 split (norm.)	bal. for C	bal. for C (norm.)	bal. for T	bal. for T (norm.)
combo 0	46.811	54.835	51.646	**54.63**	**51.646**	**52.778**
combo 1	46.193	52.058	48.971	52.16	50.206	50.309
combo 2	**53.601**	52.778	50.926	50.926	48.868	50.206
combo 3	52.469	52.778	**52.881**	49.177	50.206	51.955
combo 4	51.337	**59.156**	52.366	48.971	50.514	50.617
combo 5	48.868	48.868	49.486	51.132	48.765	51.132

Table 4: *Accuracy of the LSTM variations on the 50/50 data splits.*

most common class, i.e. sober. Interestingly, however, some of the models, namely `combo 7` for SNN and `combo 7, 10` and `11` for SNN trained on the normalised data, predicted the label "intoxicated" for a very small number of data points – for all of which this prediction was correct. Also, for the SNNs trained on the normalised data, the datapoints that were classified as "intoxicated" correctly overlap. `Combo 10` predicts three audio samples correctly, `combo 11` correctly identifies two of these and `combo 7` one of them. After listening to these three datapoints, we can say that it is in fact also possible to identify the speech as intoxicated for the human authors. What all of these three samples, and also the correctly identified sample from `combo 7` on the un-normalised dataset, have in common is that the speaker is female and sounds amused, i.e. giggling or smiling can be perceived.

The final results on the test set for the best model from each training condition, which can be found in Table 9, resemble the results obtained on the validation set for the SNNs. For the LSTM models, the results are for the most part a bit worse than on the validation set. Interestingly, performance is considerably better for the model trained on the normalised subset balanced for content and on the regular subset balanced for type.

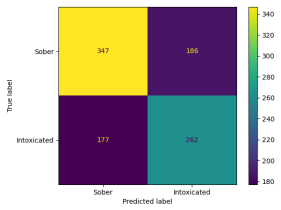

Figure 1: *Confusion matrix of the combo 3 SNN predictions on the 50/50 split.*

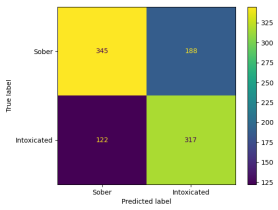

Figure 2: *Confusion matrix of combo11 SNN predictions on the norm. 50/50 split.*

5.2. Discussion

First of all, the reason for the LSTM models generally performing worse than the less complex SNN models remains unclear. It is possible, that the LLD features simply are not as important for identifying intoxicated speech as the Functional features. Further, there are more different Functional features (88) that LLD features (25).

As described in the previous section, normalisation generally helps performance, especially for the SNN models. Unfortunately, [14] do not discuss the reasons why global speaker normalisation improved their models' accuracy. A potential explanation could be that normalisation reduces the variation in speech between the speakers and therefore helps the models to focus on the features that are important in order to detect intoxication.

The reason for the mostly decreased performance when the models are trained on the balanced subsets, i.e. balanced for content or type, is most likely that this kind of balancing significantly reduced the size of the training data which can be expected to have a negative effect on the results. The fact that the models mostly just rely on predicting the most common class when trained on the unbalanced dataset, however, implies that balancing the data was an important measure to ensure that the models actually make use of the knowledge they gathered in training. Contrary to the, maybe most intuitive, assumption that the models do not learn at all when trained on the unbalanced dataset, the exceptions of the SNN models, where they predict a few datapoints correctly as "intoxicated", rather suggest that the models do gain a certain understanding of what features are important in order to identify intoxicated speech, but still resort to predicting the most common class unless they are very confident that a sample exhibits intoxicated speech.

It is further important to point out that the results that we obtained on the test dataset for the SNNs validate our results on the validation set since the similarity of the accuracy values suggest that the models do not overfit on the validation set. The LSTM results are not as clear but are also not very stable in the sense that there is no clear trend, which makes it hard to interpret the discrepancy between the results on the validation and test dataset.

Finally, the hyperparameter search has given us some insight into how some hyperparameter values or combinations can influence the performance of our models. E.g. dropout seems to be helpful only when a relatively low number of neurons are ignored in each training step. Also, the experiments suggest that good performance can be achieved when only using few linear layers which is interesting especially from a resource-oriented point of view.

6. Future Work

We acknowledge that further experiments, which could not be carried out due to time constraints, could potentially improve performance further. For instance, a more extensive hyperparameter search, that also tunes for the specific subsets of the data, could prove to be helpful, especially when combining the insights from both model architectures. Further, it would be interesting to see, if a combination of the two model architectures, and thus the two feature types, could improve performance.

7. Conclusion

In this work, we tackled the task of detecting intoxicated speech by making use of neural networks. We experimented with different preprocessing strategies as well as hyperparameter combinations and found that the SNNs we trained performed generally better than the LSTM models. Even though our models could not always beat all of our baselines, the results are still within the ball park of previous work and we could improve our results by applying global speaker normalisation and balancing the data for class labels, which gives us valuable insights that could be applied in future work.

8. References

[1] J. Liu, Y. Luo, L. Ge, W. Zeng, Z. Rao, and X. Xiao, "An intelligent online drunk driving detection system based on multi-sensor fusion technology," *Sensors*, vol. 22, no. 21, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/21/8460

[2] F. Schiel and C. Heinrich, "Disfluencies in the speech of intoxicated speakers." *International Journal of Speech, Language & the Law*, vol. 22, no. 1, 2015.

[3] S. Ultes, A. Schmitt, and W. Minker, "Attention, sobriety checkpoint! can humans determine by means of voice, if someone is drunk… and can automatic classifiers compete?" in *Twelfth Annual Conference of the International Speech Communication Association*, 2011.

[4] H. Hollien, G. DeJong, C. A. Martin, R. Schwartz, and K. Liljegren, "Effects of ethanol intoxication on speech suprasegmentals," *The Journal of the Acoustical Society of America*, vol. 110, no. 6, pp. 3198–3206, 2001.

[5] H. Bortfeld, S. D. Leon, J. E. Bloom, M. F. Schober, and S. E. Brennan, "Disfluency rates in conversation: Effects of age, relationship, topic, role, and gender," *Language and speech*, vol. 44, no. 2, pp. 123–147, 2001.

[6] B. Aishwarya, S. R. Shetty, S. Srinivas, V. Mantri, and V. B. Prasad, "Intoxication detection using audio," in *2023 IEEE 8th International Conference for Convergence in Technology (I2CT)*. IEEE, 2023, pp. 1–6.

[7] B. Schuller, A. Batliner, S. Steidl, F. Schiel, and J. Krajewski, "The interspeech 2011 speaker state challenge," in *Proc. INTERSPEECH 2011, Florence, Italy*, 2011.

[8] F. Schiel, C. Heinrich, and V. Neumeyer, "Rhythm and formant features for automatic alcohol detection," in *Eleventh Annual Conference of the International Speech Communication Association*, 2010.

[9] F. Hönig, A. Batliner, and E. Nöth, "Does it groove or does it stumble: automatic classification of alcoholic intoxication using prosodic features," 2011.

[10] C. Montacié and M.-J. Caraty, "Combining multiple phoneme-based classifiers with audio feature-based classifier for the detection of alcohol intoxication," in *Twelfth Annual Conference of the International Speech Communication Association*, 2011.

[11] T. Bocklet, K. Riedhammer, and E. Nöth, "Drink and speak: On the automatic classification of alcohol intoxication by acoustic, prosodic and text-based features," in *Twelfth Annual Conference of the International Speech Communication Association*. Citeseer, 2011.

[12] Z. Zhang, F. Weninger, and B. Schuller, "Towards automatic intoxication detection from speech in real-life acoustic environments," in *Speech Communication; 10. ITG Symposium*, 2012, pp. 1–4.

[13] F. Schiel, C. Heinrich, S. Barfüßer, and T. Gilg, "ALC: Alcohol language corpus," in *Proceedings of the Sixth International Conference on Language Resources and Evaluation (LREC'08)*. Marrakech, Morocco: European Language Resources Association (ELRA), May 2008. [Online]. Available: http://www.lrec-conf.org/proceedings/lrec2008/pdf/419_paper.pdf

[14] D. Bone, M. Li, M. Black, and S. Narayanan, "Intoxicated speech detection: A fusion framework with speaker-normalized hierarchical functionals and gmm supervectors," *Computer speech language*, vol. 28, 03 2014.

[15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[17] F. Lux, J. Koch, A. Schweitzer, and N. T. Vu, "The IMS Toucan system for the Blizzard Challenge 2021," in *Proc. Blizzard Challenge Workshop*, vol. 2021. Speech Synthesis SIG, 2021.

A. Appendix

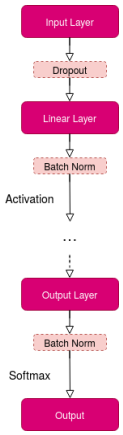

Figure 3: *Architecture of the SNN Model*

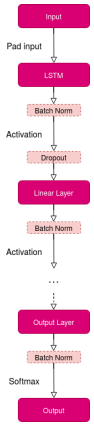

Figure 4: *Architecture of the LSTM Model*

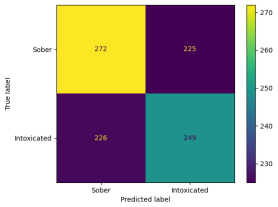

Figure 5: *Confusion matrix of the combo 2 LSTM predictions on the 50/50 split.*

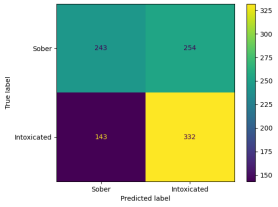

Figure 6: *Confusion matrix of combo 4 LSTM predictions on the norm. 50/50 split.*

Combo	Epochs	B. size	L.rate	Layers	Optim.	Droupout
combo 0	10	5	0.0001	[64, 32, 16, 8, 4]	Adam	False
combo 1	10	5	0.0001	[32]	Adam	False
combo 2	10	5	0.0001	[64, 16, 4]	Adam	False
combo 3	10	10	0.0001	[64, 32, 16, 8, 4]	Adam	False
combo 4	10	5	0.0001	[64, 32, 16, 8, 4]	Adam	0.5
combo 5	10	10	0.0001	[32]	Adam	False
combo 6	10	5	0.0001	[64, 32, 16, 8, 4]	SGD	False
combo 7	20	10	0.0001	[32]	Adam	False
combo 8	10	5	0.00001	[64, 32, 16, 8, 4]	Adam	False
combo 9	10	5	0.0005	[64, 32, 16, 8, 4]	Adam	False
combo 10	10	15	0.0001	[32]	Adam	False
combo 11	10	15	0.0001	[32]	Adam	0.1

Table 5: *Hyperparameter combinations for the SNN*

Combo	B. size	Layers	Droupout	Bias	Bidir.
combo 0	5	[16, 8, 4]	False	True	False
combo 1	5	[8]	False	True	False
combo 2	5	[16, 8, 4]	False	True	True
combo 3	5	[16, 8, 4]	False	False	False
combo 4	10	[16, 8, 4]	False	True	False
combo 5	5	[16, 8, 4]	0.5	True	False

Table 6: *Hyperparameter combinations for the LSTM*

Param. Combo	full data	full data (norm.)
combo 0	54.835	54.835
combo 1	54.835	54.835
combo 2	54.835	54.835
combo 3	54.835	54.835
combo 4	54.835	54.835
combo 5	54.835	54.835
combo 6	54.835	54.835
combo 7	**54.938**	54.938
combo 8	54.835	54.835
combo 9	54.835	54.835
combo 10	54.835	**55.144**
combo 11	54.835	55.041

Table 7: *Accuracy of the SNN variations trained on the full dataset and tested on the 50/50 validation split.*

Param. Combo	full data	full data (norm.)
combo 0	51.132	51.132
combo 1	51.132	51.132
combo 2	51.132	51.132
combo 3	51.132	51.132
combo 4	51.132	51.132
combo 5	51.132	51.132

Table 8: *Accuracy of the LSTM variations trained on the full dataset and tested on the 50/50 validation split.*

Param. Combo	SNN	LSTM
50/50 split	61.008	49.074
50/50 split (norm.)	**68.519**	56.687
bal. for C	56.481	51.132
bal. for C (norm.)	60.185	**56.79**
bal. for T	58.128	54.733
bal. for T (norm.)	65.021	51.029

Table 9: *Accuracy on the test set of the best hyperparameter combinations for the two architectures and each training condition.*

Train	Evaluation			
	full valid.	full valid. norm.	subset valid.	subset valid. norm.
full train	+	NA	-	NA
full train norm.	NA	-	NA	-
50-50 bal.	NA	NA	+	NA
50-50 bal. norm.	NA	NA	NA	+
speaker-content bal.	+	NA	-	NA
speaker-content bal. norm.	NA	-	NA	+
speaker-type bal.	+	NA	-	NA
speaker-type bal. norm	NA	-	NA	+

Table 10: *Further experiments with subsets*

approach	full		50-50 split		speaker-type		speaker-content	
	-	norm.	-	norm.	-	norm.	-	norm.
majority	68.906	TBD	45.165	45.165	68.906	54.835	68.906	54.835
Naive Bayes	65.876	TBD	58.539	63.374	60.738	62.963	58.564	62.963
kNN	63.834	TBD	55.761	63.272	54.348	61.420	55.731	61.420
SVM (linear)	69.236	TBD	55.350	66.152	32.082	63.786	31.225	63.889
SVM (poly)	68.906	TBD	61.626	66.152	55.731	**65.741**	68.314	65.741
Logistic Regression	70.026	TBD	61.111	66.255	62.516	63.786	61.989	63.786
Random Forest	**74.967**	TBD	**70.782**	**68.519**	**69.565**	65.329	**72.925**	**66.358**

Table 11: *Results of the ML models according to the experimental setup described in table 10*